Shoe Mechanical Degradation & Replacement Tracker

Author: **Cesar Silguero**
Course: **CS 313E**
Date: **November 11, 2026**

1. Project Idea

This program models wear and fatigue in running shoes over distances ran at different paces. In materials engineering, repetitive loading causes material degradation. The shoes absorb the impact forces with every stride and at faster speeds, the force increases.

The project takes distance and speed into account when calculating the mechanical stress "score". The score is calculated with a stress factor (10/pace) which is the inverse of pace times 10 because with smaller pace, higher speed. The stress factor is multiplied times the distance which acts as a linear factor in the score.

2. Dataset Description & Access

- File: `running_log.csv`
- Format: Comma-Separated Values (CSV).
- Access: The dataset is stored locally in the root directory of this repository alongside `shoe_stress_tracker.py`, requiring no external downloads.
- Fields:
    - `Model` (String): The brand and shoe model name.
    - `Distance` (Float): Total miles completed in the workout.
    - `Pace` (Float): Decimal minutes per mile (e.g., 8.50 = 8m 30s).
    - Irrelevant fields that are only there as part of the data

3. Software Architecture & Design

The software uses an object-oriented architecture consisting of two primary classes:

Class `Shoe`:

- **Role:** Models a single pair of shoes and accumulates mechanical load.
- **Attributes:** `model_name` (str), `mileage` (float), `stress` (float).
- **Methods:**
    - `add_run(distance, pace)`: Calculates a speed-weighted stress factor (`factor = 10 / pace`) and accumulates total distance and stress (`stress += (factor * distance)`).
    - `__str__()`: Formats and displays the shoe's name, total miles, and total stress rounded to two decimal places.

Class `Fatigue`:

- **Role:** Manages the shoe collection, passes the data file, and sorts the objects.
- **Attributes:** `inventory` (dict).
- **Methods:**
    - `load_runs(filename)`: Uses `csv.DictReader` to stream rows from the CSV file, creating new `Shoe` objects as new models appear in the file and updating existing shoe objects.
    - `sort_stress()`: Converts the inventory values into a list and runs an iterative Bubble Sort to rank the shoes in descending order of accumulated stress.

4. Libraries Used

- **csv:** Standard Python library module used to parse structured CSV files via `csv.DictReader`. No third-party packages or virtual environment installations are required.

5. Data Structures Used

Dictionary (`self.inventory` in `Fatigue`):

- Stores `shoe` profiles using the shoe model name as the key `({model_name: Shoe})`.
- Provides $O(1)$ average time complexity for checking whether a shoe model has already been logged and updating its metrics on the fly.

List (`shoes` in `Fatigue.sort_stress`):

- Holds the collection of Shoe objects extracted from the dictionary.
- Provides sequential indexing and $O(1)$ in-place element swapping required by comparison-based sorting.

6. Course Algorithm: Iterative Bubble Sort

The ranking routine in `sort_stress()` implements an iterative **Bubble Sort**:

- **Logic:** Converts the dictionary values into a list of size `n`. An outer loop runs `n` passes. An inner loop compares adjacent elements `(shoes[j].stress < shoes[j + 1].stress)` and swaps them in place if the preceding shoe has a lower stress score than the following one.
- **Complexity**:
    - **Time Complexity:** $O(n^2)$ worst and average-case.
    - **Space Complexity:** $O(1)$ auxiliary memory beyond the extracted list.
- **Outcome:** Produces a descending list placing the shoe with the greatest accumulated mechanical fatigue at index 0.

7. Test Cases and Verification

Test Case	Input	Expectation	Result
File Parsing and Aggregation	CSV containing repeated runs for “Nike Pegasus”	Multiple runs for the same model correctly aggregate into one `Shoe` instance in `self.inventory`	Passed. Unique dictionary keys and no duplicates.
Stress Weighing	Run 5 miles at 8.0 pace vs. run 5 miles at 10.0 pace	The 8.0 pace generates higher mechanical stress (factor = 1.25, stress = 6.25) than the 10.0 pace (factor = 1.00, stress = 5)	Passed. Faster paces result in higher stress factors.
Sort Correctness	Inventory with different stress values	`sort_stress()` returns models ordered strictly by descending stress scores	Passed. Shoes with highest stress appear first in the output.

8. Output Interpretation and Improvement Ideas

When processing the 30 run log, the program outputs this summary.

```
None
SHOE REPLACEMENT TRACKER
Shoe: Brooks Ghost       | Total Mileage: 66.80 | Stress:  70.72
Shoe: Nike Pegasus       | Total Mileage: 38.60 | Stress:  47.03
Shoe: Saucony Endorphin  | Total Mileage: 30.00 | Stress:  36.06
Shoe: Asics Novablast    | Total Mileage: 29.70 | Stress:  34.71
Shoe: Hoka Clifton       | Total Mileage: 19.90 | Stress:  23.33
```

- The Brooks Ghost leads the inventory of shoes because of the high stress.
- The Saucony Endorphin and Asics Novablast are very close in mileage but stress has a larger difference and is the ultimate sorting factor.

9. References

- Python Software Foundation. CSV File Reading and Writing (csv module). Python 3 Documentation.

10. How to Run the Software

Run the script from your terminal:

```
python shoe_stress_tracker.py
```